Improving a Small Language Model with SwiGLU and Rotary Position Embeddings

DASE7506 MP1 — Small Language Model Challenge

Abstract

This report investigates architectural and training improvements to the provided small GPT baseline for next-token prediction on WikiText-2. Starting from a four-layer causal Transformer with approximately 1.09M parameters, I replace the standard GELU feed-forward network with a parameter-matched SwiGLU module and then replace learned absolute positional embeddings with Rotary Position Embeddings (RoPE). Under the same training budget of 9.83M processed targets, validation bits per byte (BPB) improves from 2.0711 for the baseline to 2.0005 with SwiGLU and further to 1.8370 with SwiGLU and RoPE. Increasing the training budget of the selected architecture from 1,200 to 2,400 optimization steps further reduces validation BPB to 1.7418. The frozen final model achieves a test BPB of 1.7666 with 1,069,312 parameters. It requires approximately 1.14× the baseline CPU scoring time, 1.56 GiB peak RAM, and a 4.10 MiB checkpoint. The results show that substantial improvements in this small language model can be obtained through more effective feed-forward gating, positional representation, and additional training without increasing model size.

1. Introduction

The objective of this coursework is to improve the predictive performance of a small causal language model trained from random initialization while preserving the supplied dataset, tokenizer, evaluator, and causal prediction constraints. Model quality is measured using bits per byte (BPB), where a lower value indicates that the model assigns higher probability to the observed text.

The supplied baseline is a compact GPT-style Transformer containing four Transformer blocks, a hidden width of 128, four attention heads, a context length of 256 tokens, and 1,088,256 trainable parameters. It uses learned absolute positional embeddings and a conventional two-layer GELU feed-forward network. Using the supplied 1,200-step training configuration, I reproduced a validation BPB of 2.0711 and a complete-test BPB of 2.1013.

Rather than increasing width or depth, this work first focuses on improving how the existing parameter budget is used. Two architectural modifications are investigated: SwiGLU feed-forward networks [2] and Rotary Position Embeddings (RoPE) [3]. The baseline, SwiGLU model, and SwiGLU+RoPE model are first trained using the same 9,830,400 processed targets. After selecting the best architecture using validation performance, I separately investigate a larger 2,400-step training budget. This separates equal-budget architectural evidence from the effect of additional optimization.

2. Baseline Model

The supplied baseline is a decoder-only Transformer [1]. Each token is mapped to a learned token embedding and added to a learned absolute positional embedding. The sequence is then processed by four pre-normalized Transformer blocks. Each block consists of causal multi-head self-attention followed by a feed-forward network, with residual connections around both components.

$$h' = h + Attention(LN(h)), \quad h_out = h' + MLP(LN(h')).$$

The baseline feed-forward network expands the hidden representation from 128 to 512 dimensions, applies GELU, and projects it back to 128 dimensions:

$$MLP(x) = W_2\, GELU(W_1 x + b_1) + b_2.$$

After the four blocks, a final LayerNorm and a linear language-model head produce logits over the 2,048-token vocabulary. The output projection shares its weight matrix with the input token embedding. With seed 17, batch size 32, context length 256, and 1,200 updates, the baseline processes 9,830,400 training targets.

3. Proposed Method

3.1 SwiGLU Feed-Forward Network

The first modification replaces the conventional GELU feed-forward network with a gated SwiGLU module. Two independent linear projections are produced. One projection is passed through SiLU and used as a gate for the second projection:

$$SwiGLU(x) = W_o\,[\, SiLU(W_g\, x) \odot (W_v\, x)\,].$$

To keep the feed-forward parameter count close to the baseline, the hidden dimension is approximately parameter-matched. Ignoring biases, the baseline MLP has about $8d^2$ weights, whereas SwiGLU has about $3dh$ weights. Equating these gives $h \approx 8d/3$. For $d = 128$ this is approximately 341.3; the implementation rounds upward to a multiple of 16, giving $h = 352$. This E01 model improves validation BPB from 2.0711 to 2.0005 under the same training-target budget.

3.2 Rotary Position Embeddings

The second modification removes the learned absolute positional embedding table and applies RoPE to the query and key representations inside every attention layer [3]. Value vectors are not rotated. For each pair of head dimensions, RoPE applies a position-dependent two-dimensional rotation:

$$[x'_{2i}, x'_{2i+1}]^T = R(p\theta_i)\,[x_{2i}, x_{2i+1}]^T, \quad \theta_i = 10000^{(-2i/d_h)}.$$

Because attention scores depend on query-key interactions, these rotations inject positional structure directly into attention. Removing the learned 256×128 position table eliminates 32,768 parameters. The resulting E02 SwiGLU+RoPE model has 1,069,312 parameters, slightly fewer than the baseline.

3.3 Final Architecture

The final architecture retains four causal Transformer blocks, width 128, and four attention heads. Each block uses pre-LayerNorm, RoPE-enhanced causal self-attention, a residual connection, and the parameter-matched SwiGLU feed-forward network. E00, E01, and E02 are all trained for 1,200 steps with exactly 9,830,400 processed targets. Only after selecting E02 on validation performance is the training budget increased to 2,400 steps for E03.

4. Experimental Setup

4.1 Dataset and Evaluation Protocol

All experiments use the supplied WikiText-2 raw text [4] and the supplied train-fitted BPE tokenizer with vocabulary size 2,048. The fixed protocol is 7506-mp1-wt2-v2. Evaluation uses independent causal windows with context length 256, and every target except the first token of each split is scored once. The validation split contains 376,599 scored targets and 1,148,007 UTF-8 bytes; the test split contains 428,405 scored targets and 1,292,013 UTF-8 bytes.

$$BPB = -\Sigma_t \log_2 p(x_\{t+1\} \mid x_\{\leq t\}) \,/\, (\text{number of UTF-8 bytes}).$$

Validation is used for development and model selection. The final method is frozen before complete-test evaluation; the test result is not used to choose architectures or hyperparameters.

4.2 Training Configuration

Experiments use seed 17, batch size 32, context length 256, AdamW optimization, and the supplied training pipeline. E00–E02 each run for 1,200 steps and therefore process 1,200×32×256 = 9,830,400 targets. E03 retains exactly the E02 architecture but runs for 2,400 steps, processing 19,660,800 targets. E03 is treated as a training-budget experiment rather than an equal-budget architectural comparison.

4.3 Evaluation and Resource Measurement

Ranked evaluation is performed on CPU in FP32. In addition to BPB, I record parameter count, CPU scoring time, checkpoint size, and peak RAM. CPU scoring is compared with the baseline on the same machine. Peak CPU memory is measured externally while the supplied evaluator runs, without modifying the evaluator.

5. Results and Ablation

5.1 Equal-Budget Architectural Comparison

Run	Architecture	Steps	Targets	Parameters	Val BPB ↓
E00	Baseline	1,200	9,830,400	1,088,256	2.0711
E01	+ SwiGLU	1,200	9,830,400	1,102,080	2.0005
E02	+ SwiGLU + RoPE	1,200	9,830,400	1,069,312	1.8370

Table 1. Equal-budget architectural comparison on the validation split.

SwiGLU reduces validation BPB by 0.0706 (about 3.41%) relative to E00. Adding RoPE reduces BPB by a further 0.1635 (about 8.17%) relative to E01. Across the equal-budget sequence, BPB decreases from 2.0711 to 1.8370, an approximately 11.30% relative reduction. E02 also has about 1.74% fewer parameters than the baseline, so the improvement is not obtained by increasing model size.

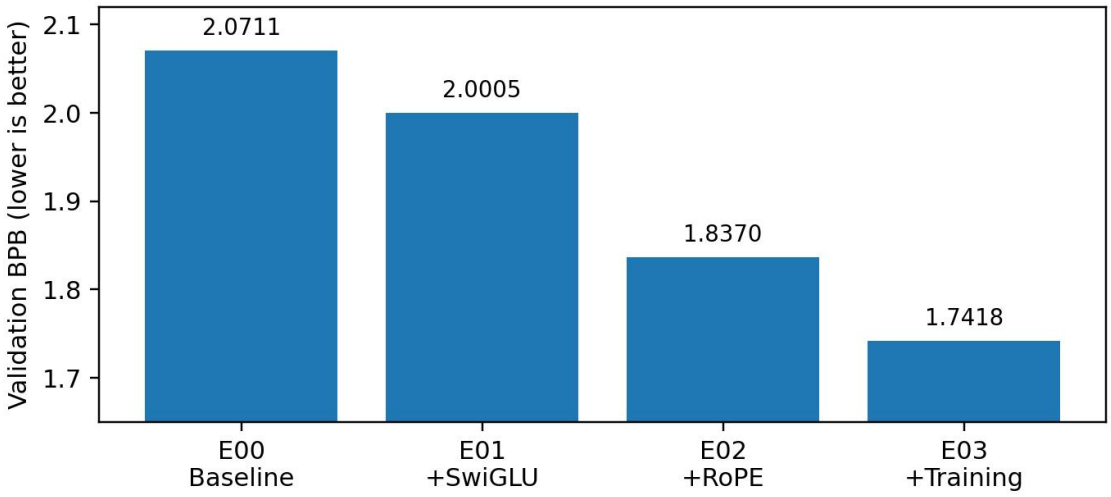


Figure 1. Validation BPB across the four main experiments.

5.2 Effect of Additional Training

Run	Architecture	Steps	Targets	Parameters	Val BPB ↓
E02	SwiGLU + RoPE	1,200	9,830,400	1,069,312	1.8370
E03	SwiGLU + RoPE	2,400	19,660,800	1,069,312	1.7418

Table 2. Effect of increasing the training budget.

Doubling the optimization budget reduces validation BPB by 0.0952, or approximately 5.18%. Because E02 and E03 have identical architectures, this improvement does not increase parameter count or architectural inference complexity, but it does increase offline training cost.

5.3 Final Test Performance

Model	Parameters	Test BPB ↓	CPU Time
Baseline	1,088,256	2.1013	18.1351 s
Final E03	1,069,312	1.7666	20.6256 s

Table 3. Frozen baseline and final-model complete-test results.

The final model reduces complete-test BPB by approximately 15.93% relative to the reproduced baseline. On the same machine, its CPU scoring time is approximately 1.14× the baseline. The checkpoint is 4,293,621 bytes (about 4.10 MiB), and measured peak CPU RAM is about 1.560 GiB. These values remain below the coursework limits of 5× baseline CPU scoring time, 4 GiB peak RAM, and 64 MiB inference assets. Repeated evaluation of the frozen checkpoint reproduced the same test BPB of 1.7666105433.

6. Discussion and Critical Analysis

6.1 Architectural Effects

The equal-budget E00–E02 experiments provide evidence that the two architectural changes are useful in this setting. SwiGLU may help because multiplicative gating allows the feed-forward layer to selectively modulate transformed features rather than applying one element-wise nonlinearity to a single projection. However, E01 has approximately 1.27% more parameters than E00, so the gain cannot be attributed exclusively to gating under perfect parameter matching.

The largest equal-budget gain occurs when learned absolute position embeddings are replaced by RoPE. A plausible explanation is that RoPE injects positional structure directly into query-key interactions and therefore into attention scores. E02 performs substantially better despite having fewer parameters than E01 and the baseline, so the gain is not explained by increased capacity. Nevertheless, the experiment establishes the performance of the implemented model under these conditions; it does not prove that relative positional structure alone causes the entire improvement.

6.2 Training Budget and Computational Cost

E03 shows that the selected architecture continues to benefit from additional optimization: doubling processed targets from 9.83M to 19.66M lowers validation BPB from 1.8370 to 1.7418. This is not an architectural improvement and should be interpreted as a trade-off between offline training cost and prediction quality. The recorded E03 wall-clock training log contains a large irregular delay between progress reports, so processed targets are a more reproducible description of the training budget than that wall-clock time.

At inference time, the final model uses slightly fewer parameters but is about 14% slower than the baseline on the measured CPU. One likely contributor is the on-the-fly sine/cosine work in the simple RoPE implementation. A precomputed fixed-context RoPE cache could potentially reduce this overhead without changing predictions, but such an optimization was not used to select the reported model.

6.3 Limitations

The experimental search is intentionally limited to two main architectural changes and one larger training budget. Other choices such as depth, width, normalization, learning-rate schedule, and regularization could be explored. Most controlled runs also use a single seed; multiple-seed repetitions would provide stronger evidence of robustness. Finally, conclusions are specific to the supplied WikiText-2 data, BPE-2048 tokenizer, context length 256, and approximately one-million-parameter scale and should not automatically be generalized to larger models or other datasets.

7. Conclusion

This work improves the supplied small GPT baseline through SwiGLU, RoPE, and additional training. Under the equal 9,830,400-target budget, validation BPB decreases from 2.0711 for the baseline to 2.0005 with SwiGLU and 1.8370 with SwiGLU+RoPE. Increasing the selected architecture from 1,200 to 2,400 steps further reduces validation BPB to 1.7418. The frozen final model achieves a complete-test BPB of 1.7666 with 1,069,312 parameters.

The final predictor uses slightly fewer parameters than the baseline and remains comfortably within the evaluation constraints: approximately 1.14× baseline CPU scoring time, 1.560 GiB peak RAM, and a 4.10 MiB checkpoint. Overall, the experiments show that improved feed-forward gating and positional representation can substantially improve prediction quality in this small-language-model setting without relying on increased model size.

References

[1]Ashish V. Attention is all you need[J]. Advances in neural information processing systems, 2017.

[2] Shazeer N. Glu variants improve transformer[J]. arXiv preprint arXiv:2002.05202, 2020.

[3]Su J, Ahmed M, Lu Y, et al. Roformer: Enhanced transformer with rotary position embedding[J]. Neurocomputing, 2024, 568: 127063.

[4]Merity S, Xiong C, Bradbury J, et al. Pointer sentinel mixture models[J]. arXiv preprint arXiv:1609.07843, 2016.

AI Assistance Disclosure

Generative AI assistance was used to help interpret the supplied coursework requirements and baseline code, discuss and implement the SwiGLU and RoPE modifications, interpret experimental results, organize reproducibility checks, and assist with drafting and editing this report. All experiments and reported outputs were executed and reviewed by the student, who is responsible for understanding and explaining the submitted method.